

1 Overview

MiniRISC is a 32-bit RISC-style processor with a 16×32 general-purpose register file and a 32-bit Program Counter. Its instruction set covers arithmetic, bitwise logic, shifts, comparisons, memory access, control flow, and constant formation, together with a multiply and multiply-accumulate instruction, implemented using a fast Booth multiplier.

Every instruction is 32 bits wide and follows one of two encodings, Register-type or Immediate-type (Section 2). Register-type instructions operate entirely on register operands (up to two sources and one destination); Immediate-type instructions carry a 16-bit immediate operand in place of a second register source, used for arithmetic against a constant, memory addressing, branch and jump displacements, and constant formation.

Both the instruction memory and the data memory are 32-bits wide, and thus word-addressable. The PC addresses the corresponding instruction word from the instruction memory, and increments on each clock.

The processor executes one instruction at a time to completion, under the control of a finite-state machine; it does not pipeline instruction execution. Most instructions complete in a fixed, small number of cycles. MUL and MAC are multi-cycle operations, since they are carried out by the Booth multiplier, and the control unit holds execution until the multiplier signals completion (Section 6).

This document specifies the instruction encodings and instruction set for MiniRISC, the register file, the memory system, the ALU, and the overall processor datapath and control architecture.

2 Instruction Format

MiniRISC instructions are 32 bits. There are two different types of instructions (and encodings):

1. Register-type instructions
2. Immediate-type instructions

2.1 Register-type instructions

These instructions operate on register operands. All register-type instructions have an opcode of 0, and they are distinguished by their fn field. The instruction format is as follows:

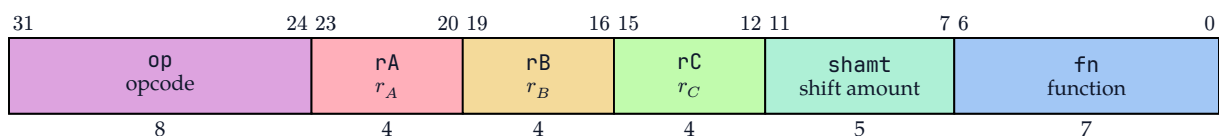


Figure 1 Instruction encoding for register-type instructions

- **opcode:** fixed 00000000 for register-type instructions
- r_A, r_B, r_C : register operands, each register is addressed by 4 bits
- **shift amount:** 5-bit immediate value representing the amount to shift by, for shift left/right instructions
- **function:** opcode extension, distinguishing the instruction to be executed

2.2 Immediate-type instructions

Instructions which require an immediate operand is categorised as an immediate-type instruction. The immediate value is 16-bits wide, which can be sign-extended / zero-extended as required, depending on the type of instruction.

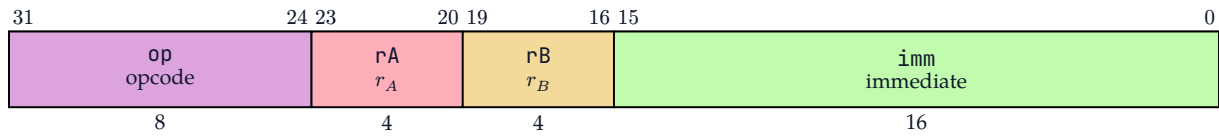


Figure 2 Instruction encoding for immediate-type instructions

- **opcode**: 8-bit identifier distinguishing the instructions
- r_A, r_B : register operands, each register is addressed by 4 bits
- **immediate**: 16-bit immediate operand

The instruction is decoded by first checking the op field, and if it is all zeroes (indicating a register-type instruction), by looking at the fn field. This gives the type of instruction to be executed.

3 Instruction Set

The following table lists the 34 instructions that MiniRISC supports, with their mnemonics, syntax, and description of what it does. Instructions with opcode 0 are register-type, and all others are immediate-type instructions. (Some instructions may not use some of their fields, in that case they are set to zero.) Note that the op field is 8 bits, and fn field is 7 bits wide. They are represented in hex.

Table 1 MiniRISC instruction set

Name	Mnemonic / Format	Description	op	fn
Add	ADD r_C, r_A, r_B	$r_C \leftarrow r_A + r_B$	0x00	0x01
Subtract	SUB r_C, r_A, r_B	$r_C \leftarrow r_A - r_B$	0x00	0x02
Multiply	MUL r_C, r_A, r_B	$r_C \leftarrow \text{low32}(r_A \times r_B)$	0x00	0x10
Multiply-accumulate	MAC r_C, r_A, r_B	$r_C \leftarrow r_C + \text{low32}(r_A \times r_B)$	0x00	0x11
Set less than	SLT r_C, r_A, r_B	$r_C \leftarrow (r_A < r_B)$	0x00	0x0E
Add immediate	ADDI r_B, r_A, imm	$r_B \leftarrow r_A + \text{SE}(\text{imm})$	0x01	—
Subtract immediate	SUBI r_B, r_A, imm	$r_B \leftarrow r_A - \text{SE}(\text{imm})$	0x02	—
Set less than immediate	SLTI r_B, r_A, imm	$r_B \leftarrow (r_A < \text{SE}(\text{imm}))$	0x0E	—
Bitwise And	AND r_C, r_A, r_B	$r_C \leftarrow r_A \wedge r_B$	0x00	0x03
Bitwise Or	OR r_C, r_A, r_B	$r_C \leftarrow r_A \vee r_B$	0x00	0x04
Bitwise Xor	XOR r_C, r_A, r_B	$r_C \leftarrow r_A \oplus r_B$	0x00	0x05
Bitwise Nor	NOR r_C, r_A, r_B	$r_C \leftarrow \sim(r_A \vee r_B)$	0x00	0x06
Bitwise Not	NOT r_C, r_A	$r_C \leftarrow \sim r_A$	0x00	0x07
Bitwise And immediate	ANDI r_B, r_A, imm	$r_B \leftarrow r_A \wedge \text{ZE}(\text{imm})$	0x03	—
Bitwise Or immediate	ORI r_B, r_A, imm	$r_B \leftarrow r_A \vee \text{ZE}(\text{imm})$	0x04	—
Bitwise Xor immediate	XORI r_B, r_A, imm	$r_B \leftarrow r_A \oplus \text{ZE}(\text{imm})$	0x05	—
Bitwise Nor immediate	NORI r_B, r_A, imm	$r_B \leftarrow \sim(r_A \vee \text{ZE}(\text{imm}))$	0x06	—
Shift left	SLL r_C, r_A, shamt	$r_C \leftarrow r_A \ll \text{shamt}$	0x00	0x08
Shift right logical	SRL r_C, r_A, shamt	$r_C \leftarrow r_A \gg \text{shamt}$ (logical)	0x00	0x09
Shift right arithmetic	SRA r_C, r_A, shamt	$r_C \leftarrow r_A \gg \text{shamt}$ (arith.)	0x00	0x0A
Shift left variable	SLLV r_C, r_A, r_B	$r_C \leftarrow r_A \ll r_B[4 : 0]$	0x00	0x0B
Shift right variable logical	SRLV r_C, r_A, r_B	$r_C \leftarrow r_A \gg r_B[4 : 0]$ (logical)	0x00	0x0C
Shift right variable arith.	SRAV r_C, r_A, r_B	$r_C \leftarrow r_A \gg r_B[4 : 0]$ (arith.)	0x00	0x0D

Move	MOVE r_C, r_A	$r_C \leftarrow r_A$	0x00	0x0F
Load word	LD $r_B, \text{imm}(r_A)$	$r_B \leftarrow \text{MEM}[r_A + \text{SE}(\text{imm})]$	0x07	—
Store word	ST $r_B, \text{imm}(r_A)$	$\text{MEM}[r_A + \text{SE}(\text{imm})] \leftarrow r_B$	0x08	—
Load immediate	LI r_B, imm	$r_B \leftarrow \text{SE}(\text{imm})$	0x0D	—
Load upper immediate	LUI r_B, imm	$r_B \leftarrow \text{imm} \ll 16$	0x0C	—
Branch (unconditional)	BR label	$\text{PC} \leftarrow \text{PC} + 1 + \text{SE}(\text{imm})$	0x0F	—
Branch if zero	BZ r_A, label	if $r_A = 0$: $\text{PC} \leftarrow \text{PC} + 1 + \text{SE}(\text{imm})$	0x09	—
Branch if minus	BMI r_A, label	if $r_A < 0$: $\text{PC} \leftarrow \text{PC} + 1 + \text{SE}(\text{imm})$	0x0A	—
Branch if plus	BPL r_A, label	if $r_A \geq 0$: $\text{PC} \leftarrow \text{PC} + 1 + \text{SE}(\text{imm})$	0x0B	—
Halt	HALT	stop execution	0xFF	—
No-op	NOP	no operation	0x00	0x00

Notes:

- The immediate field is a 16 bit value.
- SE indicates sign extension from 16 bits to 32 bits.
- ZE indicates zero extension from 16 bits to 32 bits.
- The multiplication operations only store the lower 32 bits of the result. If the multiplication results in more than 32 bits, then the result will overflow.
- All arithmetic operations are signed operations (addition, subtraction, multiplication, comparison), which consider its operands to be in the 32-bit two's complement representation.

For ease of organisation and reference, we can group the 34 instructions into the following types:

- **Arithmetic** (arithmetic operations carried out by the ALU): **ADD, SUB, MUL, MAC, SLT, ADDI, SUBI, SLTI**
 - The register versions (**ADD, SUB, MUL, MAC, SLT**) operate on r_A and r_B and write the result to r_C .
 - **MUL** and **MAC** use a fast booth multiplier which can take more than one processor cycle to complete.
 - The immediate versions (**ADDI, SUBI, SLTI**) operate on r_A and the sign-extended immediate imm , and write the result to r_B .
- **Bitwise Logic** (logical operations carried out by the ALU): **AND, OR, XOR, NOR, NOT, ANDI, ORI, XORI, NORI**
 - The register versions (**AND, OR, XOR, NOR**) operate on r_A and r_B and write the result to r_C .
 - **NOT** has a single source operand r_A and writes the result to r_C .
 - The immediate versions (**ANDI, ORI, XORI, NORI**) operate on r_A and the zero-extended immediate imm , and write the result to r_B .
- **Shift** (shift operations with constant and variable shift amounts): **SLL, SRL, SRA, SLLV, SRLV, SRAV**
 - All these instructions are register-type instructions.
 - The constant-shift versions (**SLL, SRL, SRA**) operate on r_A and use the shift amount shamt , and write the result to r_C . (r_B is unused).
 - The variable-shift versions (**SLLV, SRLV, SRAV**) operate on r_A and use the last 5 bits of r_B as the shift amount, and write the result to r_C .
- **Data Transfer** (moving data between registers and memory): **LD, ST, LI, LUI, MOVE**
 - **LD** and **ST** support indexed addressing, where the effective address in memory is $r_A + \text{imm}$.
 - **LI** and **LUI** are immediate-type instructions, which load the immediate value to r_B .
 - **MOVE** is a register-type instruction which copies the value in r_A into r_C .
- **Control Flow** (branches): **BR, BZ, BMI, BPL**
 - All these are immediate-type instructions, wherein the immediate value represents a PC relative offset to the `label` point in the program.
- **Special**: **HALT, NOP**
 - **HALT** stops the program from running.
 - **NOP** does not do anything. The all-zero instruction encoding represents a no-op.

4 Register File

The register file provides 16 general purpose 32-bit registers, R0 to R15. Each register is addressed by 4 bits (0000 to 1111 representing R0 to R15). The register file has two read ports and one write port, so it can simultaneously read from two different registers and output their values, and also write to one register. (one or more of these registers may be the same).

- R0 is hardwire to 0, so any reads from R0 result in 0. Any writes to R0 are discarded silently.

In all register-type operations, we perform reads from at most r_A and r_B , and write to at most r_C . In all immediate type operations, we perform reads from at most r_A , and write to at most r_B . So, in all cases, we require the read ports to be r_A and r_B , and the write port to be r_B or r_C depending on the instruction type. (The only exception is the **MAC** instruction where we also need to read from r_C , this can be handled by adding another mux to one of the read ports, to also read r_C .)

5 Instruction and Data Memory

5.1 Instruction BRAM ROM

The instruction memory is a **single port BRAM ROM** with a width of 32 bits, and depth of 1024. The memory is therefore word addressable, so each different address value represents a 32 bit word. To run a program (which is already written in the instruction format described above), we can initialise this ROM with the program file.

The program counter (PC) is a 32 bit register; we use its last 10 bits to address the instruction memory. Note that since the instruction memory is word-addressed, on every fetch cycle, the PC increments by 1 (not by 4).

The instruction memory supports synchronous read with a latency of one clock cycle.

5.2 Data BRAM RAM

The data memory is a **single port BRAM RAM** with a width of 32 bits and a depth of 1024 words. Again, the memory is word addressable, so load and store operations deal with 32 bit words instead of bytes.

The data memory supports synchronous read with a latency of one clock cycle. It has a write-enable control to write to the memory. (Only one of a write or a read can take place at a time.)

6 Multiplier Unit

The instructions **MUL** and **MAC** invoke a 32-bit fast Booth multiplier, which may take more than one processor cycle to compute the result. Thus, it is implemented as a sequential block, which gives a **done** signal, when turned on, indicates the completion of the computation. Hence, the control path halts the incrementing of the program counter till the operation is completed (a hardware stall).

The multiplier unit consists of an accumulator latch, a fast Booth 32-bit multiplier, and also a 32-bit adder. For a **MUL** instruction, the 32-bit multiplier is invoked, and the result is written back. For a **MAC** instruction, first the initial accumulator is loaded (r_C) and then the multiplier performs the multiplication, which is added to the accumulator and written back.

Furthermore, the **MAC** instruction involves two steps (as compared to single execution step for other instructions):

1. Fetch the old value of r_C and load it into the multiplier accumulator
2. Perform the multiplication $r_A \times r_B$ and add it to the accumulator, and write it back to r_C .

For this, we need special handling of **MAC** in the control path.

7 ALU

The ALU is a combinational block that handles all arithmetic and logical operations (other than the multiplication instructions). Control signals specify the exact operation to perform, the operations include addition, subtraction, logical operations including AND, OR, XOR, NOR, NOT, comparison by subtraction (for **SLT/SLTI**), as well as shifting units (for shift instructions and **LUI**).

Another thing that the ALU doubles as is the **effective address generation unit** for immediate-addressed operations (**LD** and **ST**). For an instruction **LD** *rB*, *imm(rA)*, it calculates the effective address as

$$EA = r_A + SE(imm)$$

which is used to address the Data BRAM.

The ALU output is also used as the register writeback for arithmetic and logical operations.